

Full Stack Architecture

React + Node.js + Express + MongoDB + Terraform + AWS scale reference for understanding and evolving this application to millions of users.

Start with frontend

Backend map

AWS scale HLD

CURRENT APP

Frontend

Vite React app with dashboard, visual AWS builder, React Flow canvas, Zustand graph state, and typed API clients.

CURRENT APP

Backend

Express API with JWT auth, route groups, Mongoose models, AWS SDK integrations, Terraform generation, and deployment runner.

CURRENT APP

MongoDB

Workspace-scoped collections store users, diagrams, AWS accounts, deployments, audit logs, and agent conversations.

How to use this PDF

Use the top chips and bookmarks to jump between architecture layers. Each diagram maps to files already present in this repository and then extends them into production-ready AWS design choices.

Output goal

Millions-user target

Move to CloudFront, S3, ECS or EKS, queues, workers, MongoDB Atlas, observability, and multi-AZ AWS networking.

Frontend Architecture

How the Vite React client is structured and how data moves through the dashboard.



Frontend module responsibilities

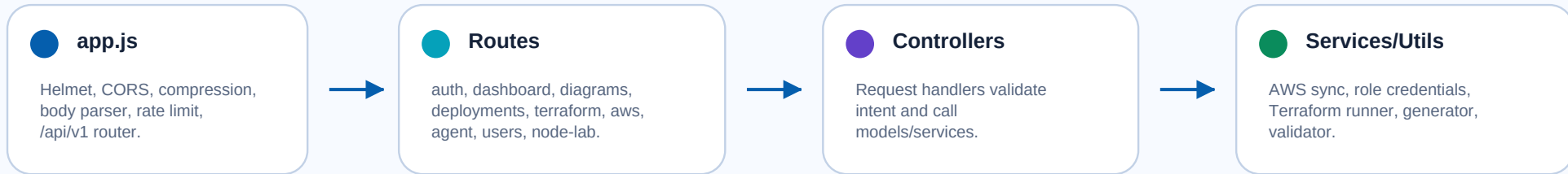
src/auth	Auth pages, token storage, login/register API client.
src/dashboard	Dashboard pages, AWS API client, diagram API client, runtime lab.
src/components	Canvas, toolbar, sidebar, deployment modal, custom React Flow nodes.
src/store	Zustand diagram state, history, clipboard, undo/redo.
src/utills	Terraform export, validation, import parsing, deployment API.

Frontend LLD focus

State	Zustand stores diagram graph. React hooks store page UI state.
API	Clients send bearer JWT for protected dashboard APIs.
UX	React Flow powers drag, connect, selection, export, and deploy.
Scale	Use CDN caching and static asset delivery.

Backend Architecture

Express, route groups, controllers, services, utilities, and operational boundaries.



API capability map

Auth	JWT, bcrypt, user/workspace creation, refresh/logout/me.
AWS	Account connection, STS identity, live sync, Cost Explorer, CloudTrail, inventory.
Diagram	Workspace-scoped visual graph persistence and validation.
Deployment	Plan creation, Terraform generation, apply/queue lifecycle, logs.
Runtime Lab	Authenticated Node demo route for processes, workers, CPU cores.

Backend production boundary

Keep API stateless

JWT auth, workspace IDs, and MongoDB persistence let API containers scale behind an ALB.

Move long work out

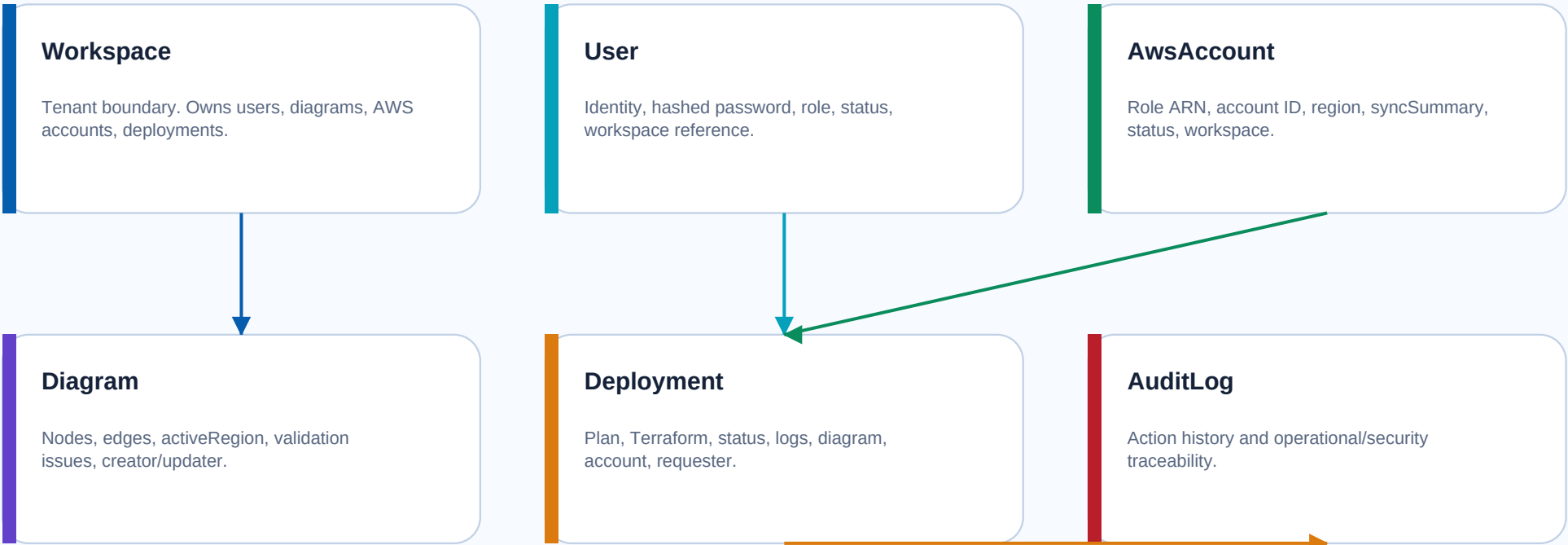
Terraform apply, AWS sync, graph analysis, and AI jobs should move to queue workers.

Use service boundaries

API -> workers -> MongoDB -> S3.

MongoDB Data Architecture

Tenant-scoped documents and relationships for Node, React, and AWS workflows.



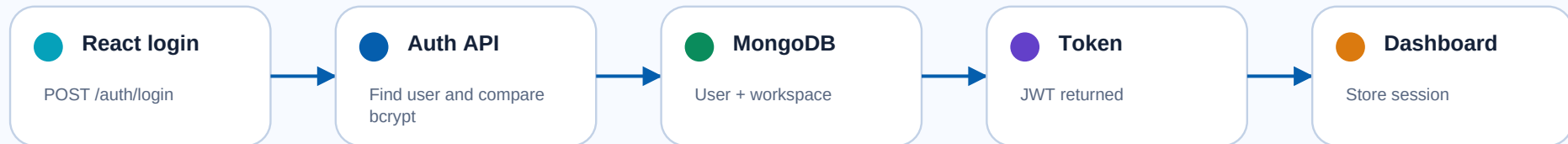
Index and scale guidance

workspace	Index workspace on tenant-owned collections for isolation and fast lists.
awsAccount	Unique workspace + accountId prevents duplicate connected accounts.
deployments	Index workspace + createdAt/status for queues, history, and polling.
large graphs	Move large Terraform artifacts/logs to S3 when documents grow.

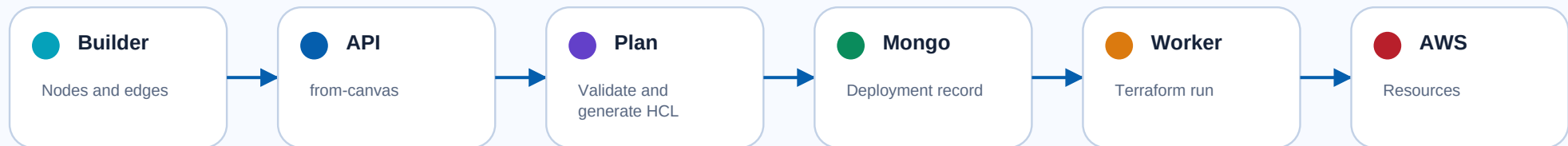
Critical Runtime Flows

End-to-end flows that connect React UI, Node API, MongoDB, AWS, and Terraform.

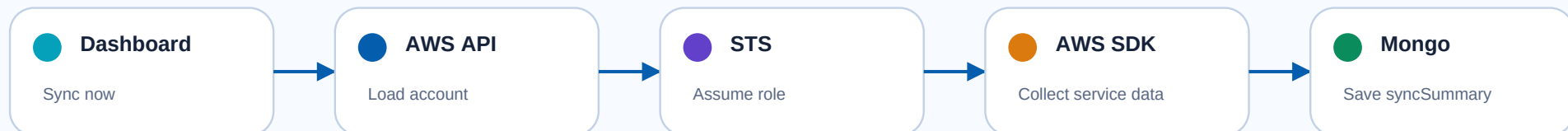
Login flow



Deployment flow

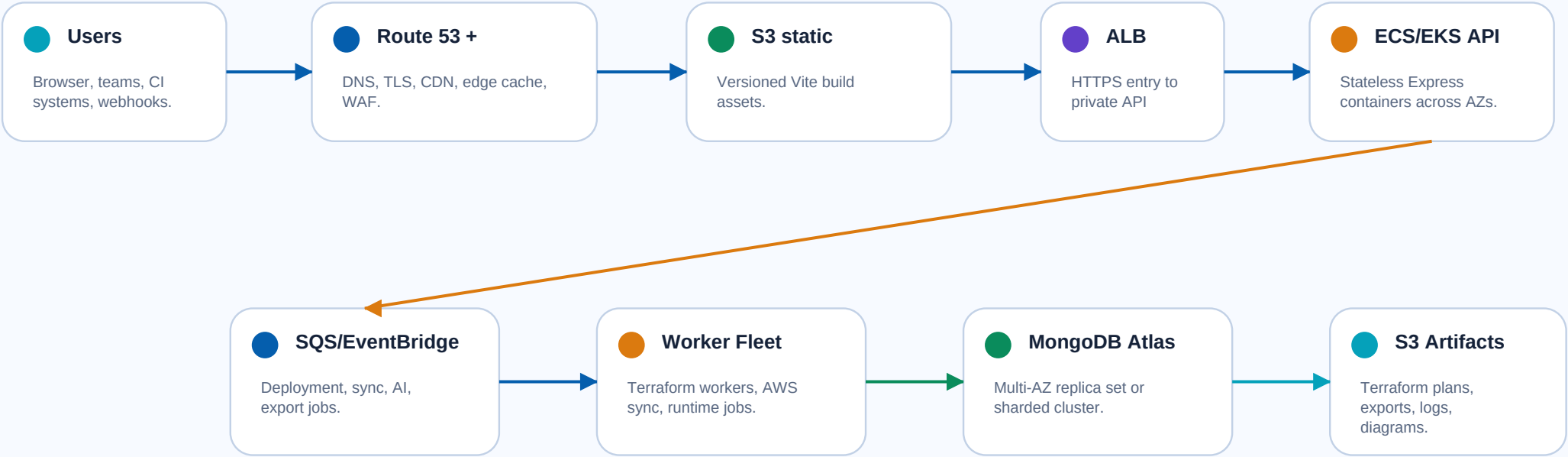


AWS sync flow



HLD For Millions Of Users On AWS

Target cloud architecture for high availability, scale, and operational separation.



Availability design

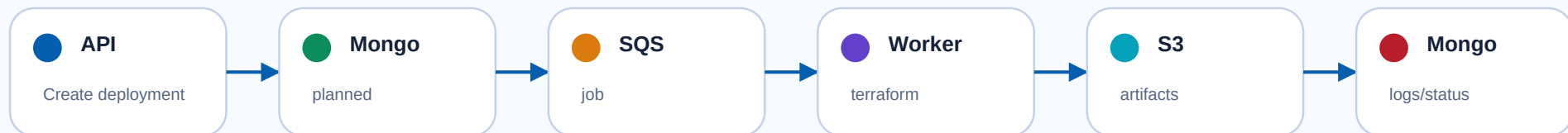
Frontend	CloudFront + S3 gives global static scale.
API	Multiple containers across private subnets behind ALB.
Jobs	SQS decouples long Terraform and AWS sync work.
Data	Atlas replica set/shards, backups, point-in-time restore.

LLD For Production Implementation

Concrete services, interfaces, queues, indexes, and runtime decisions.

API container	Express app, no local state, health/readiness endpoints, autoscale on CPU/RPS.
Worker container	Consumes SQS jobs, runs Terraform, AWS sync, graph analysis, AI tasks.
Job schema	jobId, workspaceId, type, payload S3 key, status, attempts, timestamps.
Mongo indexes	workspace+createdAt, workspace+status, workspace+accountId, user email unique.
Artifacts	Store Terraform HCL, tfplan, logs, exports, large payloads in S3.
Realtime	Use polling now; upgrade to WebSocket/AppSync/SSE for live deployment logs.
Secrets	JWT secrets and AWS app secrets in Secrets Manager or SSM Parameter Store.

LLD sequence: deployment job



AWS Scaling Blueprint

Service choices for growing the current MERN app to millions of users.



Autoscaling rules

API	Scale on ALB request count, p95 latency, CPU, memory.
Workers	Scale on SQS queue depth and oldest message age.
Mongo	Scale Atlas cluster tier, read replicas, then shard by workspace.
CloudFront	Cache static assets aggressively; invalidate only versioned build paths.

Security, Reliability, And Operations

Controls required before this app handles enterprise-scale users and AWS accounts.

Security

Auth

JWT, refresh rotation, SSO later.

Tenant

Workspace filters on queries.

Secrets

Secrets Manager and KMS.

AWS

External ID and least privilege.

Reliability

Multi-AZ

ALB, ECS, workers, Atlas HA.

Retries

Idempotent jobs and DLQs.

Backups

Atlas PITR, S3 versioning.

Limits

Rate limits and backpressure.

Observability

Logs

JSON logs per request/job.

Metrics

RPS, latency, queue depth.

Tracing

OpenTelemetry across services.

Audits

Deployment and AWS actions.

Production guardrails for Terraform

Use remote S3 state with DynamoDB locking, plan approval, separate worker IAM role, per-workspace isolation, artifact encryption, job cancellation, and explicit destroy approval before enabling full self-service infrastructure changes.

Implementation Roadmap

Practical migration path from the current repo to a scalable AWS production platform.

Phase 1 - Stabilize current app

Auth-gate all dashboard APIs, add tests, validate env, keep Terraform apply disabled by default, add Vite proxy and deploy checks.

Phase 2 - Split long-running jobs

Introduce SQS, worker service, deployment job records, S3 artifacts, and status polling for Terraform and AWS sync.

Phase 3 - Production AWS baseline

CloudFront/S3 frontend, ALB/ECS API, ECS workers, MongoDB Atlas, Secrets Manager, WAF, CloudWatch.

Phase 4 - Enterprise scale

SSO, RBAC policies, sharded MongoDB, Redis caching, OpenTelemetry, WebSocket logs, multi-region DR.

Most important design change

Separate request/response from execution

For millions of users, the API must quickly accept work, persist it, enqueue it, and return. Workers perform slow jobs and stream/persist status. This single change unlocks horizontal scaling and better reliability.